



**FACULTAD  
DE INGENIERIA**

---

Universidad de Buenos Aires

# Trabajo Práctico

2° Cuatrimestre 2026

**Ingeniería del Software I — FIUBA**

**Tema: Matrimillas**

*Economía de puntos entre personas*

Cronograma y modalidad de trabajo

# Índice

|                                                     |           |
|-----------------------------------------------------|-----------|
| Matrimillas: Economía de puntos entre personas..... | 3         |
| <b>Modalidad de Trabajo.....</b>                    | <b>4</b>  |
| <b>Objetivos.....</b>                               | <b>5</b>  |
| Reglas de negocio:.....                             | 5         |
| Cálculo y valuación de acciones.....                | 5         |
| Evidencia y aprobación.....                         | 5         |
| Uso y límites de acciones.....                      | 6         |
| Economía y saldos.....                              | 6         |
| Engagement y experiencia de grupo.....              | 6         |
| Roles y administración.....                         | 7         |
| Ciclo de vida del grupo.....                        | 7         |
| Notificaciones.....                                 | 7         |
| <b>Herramientas a utilizar / Restricciones.....</b> | <b>8</b>  |
| <b>Entorno.....</b>                                 | <b>8</b>  |
| <b>Cronograma.....</b>                              | <b>8</b>  |
| <b>Entregables.....</b>                             | <b>9</b>  |
| Sprint 0.....                                       | 9         |
| Cada sprint.....                                    | 9         |
| Product Owner.....                                  | 9         |
| Scrum Master.....                                   | 9         |
| Desarrollador.....                                  | 10        |
| Tester / QA.....                                    | 10        |
| Equipo en conjunto.....                             | 10        |
| Final del Tp.....                                   | 12        |
| <b>Criterios de corrección.....</b>                 | <b>13</b> |
| Sprint 0.....                                       | 13        |
| Cada sprint.....                                    | 13        |

## Matrimillas: Economía de puntos entre personas

**Matrimillas** es una aplicación pensada para parejas, familias o grupos de amigos que buscan repartir de forma más transparente la carga de tareas cotidianas. Su funcionamiento se basa en una **economía de puntos** —las "**matrimillas**"—, donde algunas acciones acreditan puntos (cortar el pasto suma 5, por ejemplo) y otras los consumen (salir con amigos cuesta 50). La plataforma ofrece un catálogo de acciones predefinido, que cada grupo puede ampliar con sus propias acciones custom según su dinámica particular.

El valor de cada acción no es fijo, sino que se define por votación de todos los usuarios de la plataforma: se promedia lo que vota cada uno y se actualiza cada hora, de forma similar a la cotización de una acción bursátil. Sobre ese valor base pueden aplicarse además modificadores variables según el contexto —por ejemplo, que salir a comer valga 10 en general, 5 un viernes y 20 un lunes—. A esto se suma un sistema de transferencias entre usuarios, préstamos, intereses, historial de movimientos, límites y estadísticas de uso.

Para que una acción acredite puntos, quien la realiza debe subir una evidencia (una foto, por ejemplo) que el resto del grupo tiene que aprobar. Es a partir de esta validación que se despliega el resto de las reglas de negocio, dándole cuerpo y complejidad a la solución.

# Modalidad de Trabajo

Se organizarán en grupos de 6 alumnos. Trabajaremos dentro del marco Scrum, con sprints de 2 semanas. La duración total del desarrollo será de 9 semanas: un "sprint 0" de una semana + 4 sprints de 2 semanas.

Durante el sprint 0, todos los miembros del equipo se encargarán de entender el problema mediante herramientas de product discovery.

Luego, durante cada sprint, con el fin de practicar las diferentes facetas de la ingeniería de software, los miembros del grupo deberán rotar entre los siguientes roles:

## Product Owner (PO)

- Refina el backlog: arma historias de usuario y criterios de aceptación.
- Prioriza historias para el sprint con input del equipo.

## Scrum Master (SM)

- Facilita y lleva a cabo las reuniones: planning, daily, retro.
- Se asegura de que se cumpla el proceso (definition of done, flujo de git, etc.).

## Desarrollador

- Implementa las funcionalidades del sprint, aplicando clean code y la arquitectura definida.

## Tester / QA

- Redacta y ejecuta casos de prueba según criterios de aceptación.
- Ejecuta pruebas manuales y automáticas antes de hacer merge.
- Reporta bugs como issues en Jira.
- Verifica la corrección de bugs.

**Aclaraciones:** Todos los miembros del equipo deben desarrollar en todos los sprints, independientemente del rol que tengan. Una misma persona no puede repetir 2 veces el mismo rol durante el tp, a excepción del rol "desarrollador".

Ejemplo:

**Valido:** Dev, PO, QA, Dev

**Invalido:** PO, Dev, PO, SM

## Objetivos

- Validar correctamente el problema en el sprint 0.
- Crear un proyecto en Jira para trabajar en grupo.
- Cargar todas las historias en Jira y estimarlas en story points usando planning poker.
- Armar Documento de Arquitectura preliminar basado en 4+1 y definir atributos de calidad.
- Definir junto al ayudante el alcance a implementar, MVP o Early Testable Product, para trabajar en 4 sprints de 2 semanas.
- Definir flow a utilizar y forma de trabajo en grupo.
- Implementar una versión de la API con persistencia en base de datos, basada en el template.
- Armar una colección de Postman o Swagger con las invocaciones a cada endpoint de la API.
- Cubrir el 80% del código del backend con tests automatizados.
- Modelar e implementar el sistema que sea configurable y fácilmente extensible.
- Implementar una UI muy básica que permita usar el sistema.
- Aplicar las técnicas vistas en la teoría y en la práctica.
- Realizar entrega de aceptación TP con demo a acordar con el ayudante.

## Reglas de negocio:

Estas reglas de negocio son de carácter obligatorio: cada grupo debe elegir e implementar **15** de esta lista, a su criterio, según lo que mejor se adapte a su solución. Cuantas más reglas de esta lista implementen **mejor se pondera en la nota final**.

Además, el grupo puede incorporar **reglas de negocio propias**, surgidas de investigación propia o de decisiones de diseño que consideren relevantes para el problema. Estas **también suman a la evaluación**, siempre que queden debidamente detalladas y justificadas en el informe final.

## Cálculo y valuación de acciones

- **Motor de reglas combinables:** el valor final de una acción no depende solo del valor base, sino de una serie de reglas contextuales que se van aplicando en cadena (día de la semana, feriado, urgencia declarada por quien pide la acción, racha del grupo, etc.), cada una sumando o multiplicando sobre el resultado de la anterior.
- **Congelamiento de valor:** durante los últimos 5 minutos de cada hora, la cotización de una acción no puede modificarse, para evitar que se manipule justo antes del cierre de la votación horaria. Cuando se cumple la hora, se recalcula el valor con lo votado y se vuelve abrir para el siguiente ciclo.
- **Combos:** se otorga un bonus adicional cuando se completan varias acciones relacionadas dentro de la misma semana.
- **Bonus por fecha especial:** en fechas puntuales que el grupo defina (cumpleaños, aniversario, feriado especial), ciertas acciones valen un plus fijo sobre su valor habitual.

## Evidencia y aprobación

- **No autoaprobación:** quien sube la evidencia de una acción no puede ser quien la aprueba.
- **Aprobación configurable:** cada grupo define cómo se aprueba una acción — alcanza con que la apruebe una sola persona, tienen que estar de acuerdo todos, o decide un "árbitro" que va rotando

entre los miembros.

- **Revisión ante rechazo:** si una acción es rechazada, se puede pedir que la revise otro integrante del grupo antes de descartar definitivamente.
- **Vencimiento automático:** una acción pendiente de aprobación por más de N días queda "expirada" — no acredita puntos ni se rechaza, queda en un estado propio.
- **Tipo de evidencia configurable por acción:** cada acción (del catálogo o custom) define qué tipo(s) de evidencia acepta (foto, video, comprobante, ubicación, ninguna) y si es obligatoria u opcional. Por ejemplo, se podría exigir evidencia obligatoria **solo a partir de cierto valor en puntos**, y dejar las acciones chicas bajo sistema de honor.
- **Plazo de carga de evidencia:** la evidencia debe subirse dentro de un plazo determinado desde que se marca la acción como realizada; pasado ese plazo, la acción no puede acreditarse.
- **Penalización por rechazos reiterados:** si a un usuario le rechazan varias evidencias seguidas en un período corto, queda temporalmente "bajo revisión" (por ejemplo, requiere doble aprobación hasta recuperar confianza).

### Uso y límites de acciones

- **Cooldown:** una acción no puede acreditarse más de N veces por día o por semana, para evitar que se convierta en una fuente ilimitada de puntos.
- **Ciclo de vida de acciones custom:** si una acción custom no se usa durante un período de tiempo determinado, se archiva y deja de participar de la votación global de valores.
- **Acciones colaborativas:** una acción puede requerir la participación de 2 o más personas para acreditarse, repartiendo los puntos entre quienes efectivamente colaboraron.

### Economía y saldos

- **Límite de saldo negativo:** existe un tope máximo de puntos que un usuario puede deber.
- **Vencimiento de puntos:** los puntos acumulados y no utilizados caducan luego de un período determinado, para incentivar su uso.
- **Conversión entre economías de distintos grupos:** si un usuario pertenece a más de un grupo, puede convertir matrillitas entre ambos según una tasa de cambio propia de cada par de grupos.
- **Marketplace de acciones custom entre grupos:** un grupo puede licenciar una acción custom que creó a otro grupo, cobrando una comisión cada vez que ese otro grupo la usa. Implica manejar la propiedad de la acción, tasas de comisión y liquidación entre grupos.

### Engagement y experiencia de grupo

- **Favoritos:** cada usuario puede marcar acciones de uso frecuente para registrarlas más rápido, sin afectar las reglas de negocio en sí.
- **Ranking del grupo:** los integrantes pueden ver un leaderboard con el saldo de puntos de cada uno dentro del grupo.

## Roles y administración

- **Rol de administrador:** un rol de administrador a nivel plataforma, separado de los roles internos de cada grupo (arbitro, aprobador , etc). No pertenece a ningún grupo en particular, sino que opera sobre el sistema.
- **Suspensión/reactivación de usuarios:** el administrador puede suspender o reactivar la cuenta de un usuario (por ejemplo, ante uso indebido o reportes de otros usuarios), bloqueando temporalmente su capacidad de operar en la plataforma.
- **Estadísticas globales:** el administrador puede consultar estadísticas agregadas de uso de la plataforma (grupos activos, acciones más usadas, volumen de puntos en circulación, etc.), sin acceder al detalle de movimientos privados de cada grupo.
- **Auditoría de acciones administrativas:** toda acción que realice un administrador (suspender un usuario, modificar el catálogo, etc.) queda registrada en un log auditable, indicando quién la hizo y cuándo.

## Ciclo de vida del grupo

- **Invitación por link o código:** para sumarse a un grupo, un usuario necesita un link de invitación o un código que le comparta alguien que ya pertenezca a ese grupo; no hay alta libre a un grupo existente.
- **Creación de un grupo:** cualquier usuario puede crear un grupo nuevo, convirtiéndose automáticamente en su primer integrante y definiendo la configuración inicial.
- **Salida de un grupo:** un integrante puede abandonar un grupo voluntariamente; hay que definir qué pasa con su saldo.
- **Disolución de un grupo:** si el grupo se disuelve (o queda sin integrantes), hay que definir qué pasa con los saldos pendientes de cada uno y con las acciones custom que ese grupo haya creado.
- **Expulsión de un integrante:** el grupo puede remover a un integrante (por votación o decisión del árbitro), con las mismas consideraciones de saldo que la salida voluntaria.

## Notificaciones

- **Notificación de aprobación/rechazo:** cuando se aprueba o rechaza una acción, la persona que subió la evidencia recibe una notificación con el resultado.
- **Notificación de solicitud de aprobación:** cuando alguien sube evidencia que requiere aprobación, se notifica a quien tiene que aprobar.
- **Resumen periodico del grupo:** cada integrante recibe un resumen con la actividad del grupo, acciones , ranking , y cambios de cotización.
- **Recordatorio de evidencia pendiente:** si una acción le queda poco tiempo antes de vencer su plazo de carga de evidencia , se notifica a quien la marcó como realizada.
- **Alerta de saldo negativo:** cuando el saldo de un usuario se acerca al límite máximo de deuda permitido, el sistema le envía un aviso.

## Herramientas a utilizar / Restricciones

- Jira
- Java
- Unit testing
- Git / GitLab
- Docker / Docker-Compose
- Auth: Para consumir la API, el cliente deberá autenticarse primero para obtener un JWT, que luego enviará como token en las llamadas siguientes.

## Entorno

Se deberá desarrollar el trabajo práctico sobre la base entregada por la cátedra. Los desarrollos realizados deberán pasar el pipeline de CI/CD.

Cada grupo tendrá asignada una URL del tipo <https://grupo-xx.tp1.ingsoft1.fiuba.ar/>

## Cronograma

- **Sprint 0:** 14-sep al 21-sep
  - **Revisión de avance:** 21-sep
- **Sprint 1:** 21-sep al 7-oct
  - **Demo sprint 1:** 7-oct
- **Sprint 2:** 7-oct al 21-oct
  - **Demo sprint 2:** 21-oct
- **Sprint 3:** 21-oct al 4-nov
  - **Demo sprint 3:** 4-nov
- **Sprint 4:** 4-nov al 18-nov
  - **Demo sprint 4:** 18-nov
- **Demo final/Aceptación TP:** 25-nov o 30-nov

# Entregables

## Sprint 0

- Resultados de las entrevistas.
- Mapas de empatía de las personas afectadas por el problema.
- User personas de las personas afectadas por el problema.
- Conclusiones sobre el problema relevado y propuesta de solución.
- Primeras epicas e historias de usuario, a grandes rasgos. Tablero de Jira armado y listo para trabajar.
- (Opcional) Alguna idea sobre cómo se ve la solución propuesta. Wireframes, flujos, etc.

## Cada sprint

Cada entregable debe existir al momento de la demo. Lo que no esté en el lugar indicado se considera no entregado.

*"Documentación del grupo" es el lugar que cada equipo elija para su documentación (wiki de GitLab, carpeta /docs del repo, Google Drive, etc.). El único requisito es que sea uno solo, esté indicado en el README y sea accesible para el ayudante.*

## Product Owner

| Entregable                                  | Dónde queda             | Se corrige verificando que...                                                                                                             |
|---------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Meta del sprint                             | Jira                    | Sea una oración concreta y todas las historias del sprint aporten a ella.                                                                 |
| Historias de usuario del sprint             | Jira                    | Sigan el formato "Como... quiero... para...", tengan criterios de aceptación claros, estimación en story points y responsables asignados. |
| Backlog priorizado                          | Jira                    | El orden refleje prioridad real (no orden de carga) y las épicas estén vinculadas a sus historias.                                        |
| Registro de cambios de alcance              | Documentación del grupo | Está escrito qué se sacó o agregó respecto de lo planificado y por qué.                                                                   |
| Recuerde que el PO también es desarrollador |                         |                                                                                                                                           |

## Scrum Master

| Entregable          | Dónde queda                              | Se corrige verificando que...                                                           |
|---------------------|------------------------------------------|-----------------------------------------------------------------------------------------|
| Acta de planning    | Documentación del grupo (una por sprint) | Incluya fecha, asistentes, meta acordada, historias comprometidas y capacidad estimada. |
| Registro de dailies | Documentación del grupo                  | Haya un registro breve de cada daily con los bloqueos identificados.                    |

| Entregable                                         | Dónde queda                              | Se corrige verificando que...                                                                                                          |
|----------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Registro de impedimentos</b>                    | Documentación del grupo                  | Cada impedimento tenga fecha de detección, responsable y fecha de resolución (o estado abierto).                                       |
| <b>Burndown del sprint</b>                         | Documentación del grupo (uno por sprint) | Exista, sea consistente con el tablero y el SM pueda explicar los desvíos.                                                             |
| <b>Team velocity</b>                               | Documentación del grupo                  |                                                                                                                                        |
| <b>Retrospectiva</b>                               | Documentación del grupo (una por sprint) | Tenga acciones de mejora concretas con responsable, y una verificación explícita de si se cumplieron las acciones del sprint anterior. |
| <b>Cumplimiento del proceso</b>                    | Repositorio                              | Los PRs respeten el flujo de Git acordado y la definition of done que el equipo definió.                                               |
| <b>Recuerde que el SM también es desarrollador</b> |                                          |                                                                                                                                        |

## Desarrollador

| Entregable                               | Dónde queda                             | Se corrige verificando que...                                                                                                            |
|------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Commits</b>                           | GitLab                                  | Sean atómicos, con mensaje descriptivo y referencien la historia de Jira (ej. MAT-12: ...).                                              |
| <b>Pull requests</b>                     | GitLab                                  | Cada PR corresponda a una historia, tenga descripción, pase el pipeline y tenga al menos una revisión aprobada de otro integrante.       |
| <b>Funcionalidad en producción</b>       | URL del grupo                           | Todo lo marcado como "Done" en Jira esté desplegado y funcione en la demo.                                                               |
| <b>Decisiones de diseño documentadas</b> | Repositorio (ADRs o sección del README) | Toda decisión relevante (patrón aplicado, cambio de arquitectura) tiene un registro corto: contexto, decisión, alternativas descartadas. |

## Tester / QA

| Entregable                                             | Dónde queda                              | Se corrige verificando que...                                                                                                |
|--------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Casos de prueba</b>                                 | Documentación del grupo o Jira           | Cada historia del sprint tenga sus casos, trazables a los criterios de aceptación.                                           |
| <b>Tests automatizados</b>                             | Repositorio                              | Cada historia "Done" tenga tests que la respalden y el coverage del backend se mantenga o suba respecto del sprint anterior. |
| <b>Reporte de ejecución</b>                            | Documentación del grupo (uno por sprint) | Indique qué casos se ejecutaron, con qué resultado, y el coverage al cierre del sprint.                                      |
| <b>Bugs reportados</b>                                 | Jira                                     | Tengan pasos para reproducir, resultado esperado vs. obtenido y severidad.                                                   |
| <b>Recuerde que el Tester también es desarrollador</b> |                                          |                                                                                                                              |

## Equipo en conjunto

| Entregable                     | Dónde queda            | Se corrige verificando que...                                                                                              |
|--------------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Incremento del producto</b> | Ambiente de producción | Todo lo demostrado corra sobre la URL del grupo, no en branches ni ambientes locales.                                      |
| <b>Demo</b>                    | Fecha del cronograma   | Cualquier integrante pueda presentar y responder preguntas sobre cualquier parte del trabajo.                              |
| <b>README actualizado</b>      | Repositorio            | Permita levantar el proyecto localmente siguiendo solo las instrucciones, e indique dónde está la documentación del grupo. |

## Final del Tp

- Informe final con
  - Análisis del problema: Todo el relevamiento del sprint 0.
  - Arquitectura: basada en 4+1 y atributos de calidad.
  - Modalidad de trabajo, flow a usar para los commits.
  - Documentación que fueron generando en los sprints (reuniones, métricas, velocidad del equipo)
  - Post-mortem (cosas a mejorar, potenciales adiciones, etc).
  - Conclusiones: Hablar un poco de algunas decisiones de diseño (porque hicimos tal cosa así y no de otra forma, porque decidimos que esto es lo mejor, etc).
- Trabajo finalizado (basado en lo acordado con cada ayudante), desplegado en ambiente de producción.

# Criterios de corrección

## Sprint 0

- Asistencia obligatoria para la revisión de avance al final del sprint 0.
- Cada integrante del equipo debe contribuir al product discovery. Se harán preguntas individuales y se evaluarán sus respuestas.

## Cada sprint

- La asistencia a demos/reviews es obligatoria para todos los integrantes del grupo. Máximo una falta sobre el total de 4 demos.
- Cada demo debe ser presentada sobre el ambiente de producción. No se aceptan funcionalidades dispersas sobre diferentes branches o ambientes.
- Cada integrante del equipo debe contribuir al progreso del trabajo práctico. Se harán preguntas individuales y se evaluarán los entregables según el rol correspondiente.
- Cada integrante del equipo debe poder presentar en las demos y estar al tanto del estado del trabajo práctico.
- Jira:
  - Todas las historias deben estar bien definidas, tener criterios de aceptación claros, estimación y asignación a un responsable.
  - El tablero debe reflejar el estado real del trabajo en todo momento.
- Código:
  - Se deben aplicar buenas prácticas de código limpio y principios SOLID.
  - Donde corresponda, aplicar patrones de diseño.
  - Los commits deben ser atómicos y con mensajes descriptivos.
- Tests:
  - Toda funcionalidad completa debe tener tests que la respalden.
- Documentación:
  - El README debe estar actualizado e incluir instrucciones para levantar el proyecto localmente.
  - Se deben documentar las decisiones arquitectónicas relevantes.